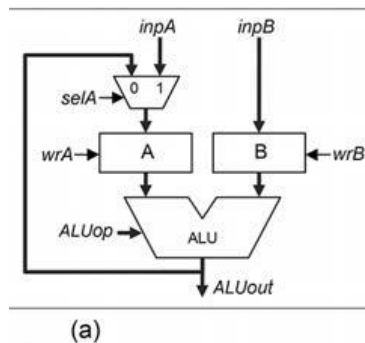


Examen du Cours CE419	Première session année 2024 – 2025 Filière EIS App Durée : 40 mn
Nom : Prénom :	Tout document autorisé

Les nombres entre crochets [X] après les questions indiquent approximativement le temps en minute à consacrer à chaque question ; ces nombres donnent également une indication du barème.

Contrôle d'un chemin de données [40]

Supposons que le chemin de données de la figure (a) doit fonctionner comme un circuit d'addition et d'accumulation (ACC), **accumulant dans A quatre valeurs consécutives de inpB**. L'impulsion de validation des données (**dv**), qui ne dure qu'une période d'horloge, doit déclencher cette procédure, après quoi **le résultat doit rester affiché dans ALUout** jusqu'à ce qu'une nouvelle impulsion **dv** se produise.



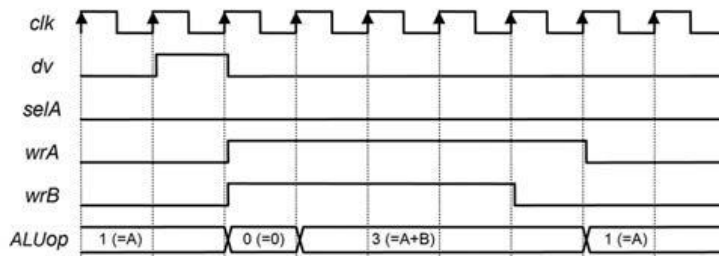
ALUop	ALUout
0	zero
1	A
2	B
3	A + B

(b)

Remarques :

- la figure (b) décrit les opérations réalisées par l'ALU et la valeur sur ALUout en fonction de la commande ALUop
- les registres A et B capturent leur entrée quand leur commande wrA et wrB sont à '1'

Voici le chronogramme correspondant à ce fonctionnement :



1) Avec des valeurs de *inpA* et *inpB* respectivement de 1 et 3, complétez les valeurs de *inpA*, *inpB*, *A*, *B* et *ALUout* sur le chronogramme de l'annexe 1 [12].

2) Dessinez un diagramme de transitions correspondant à la machine de contrôle de Moore contrôlant ce circuit pour obtenir ce fonctionnement [12].

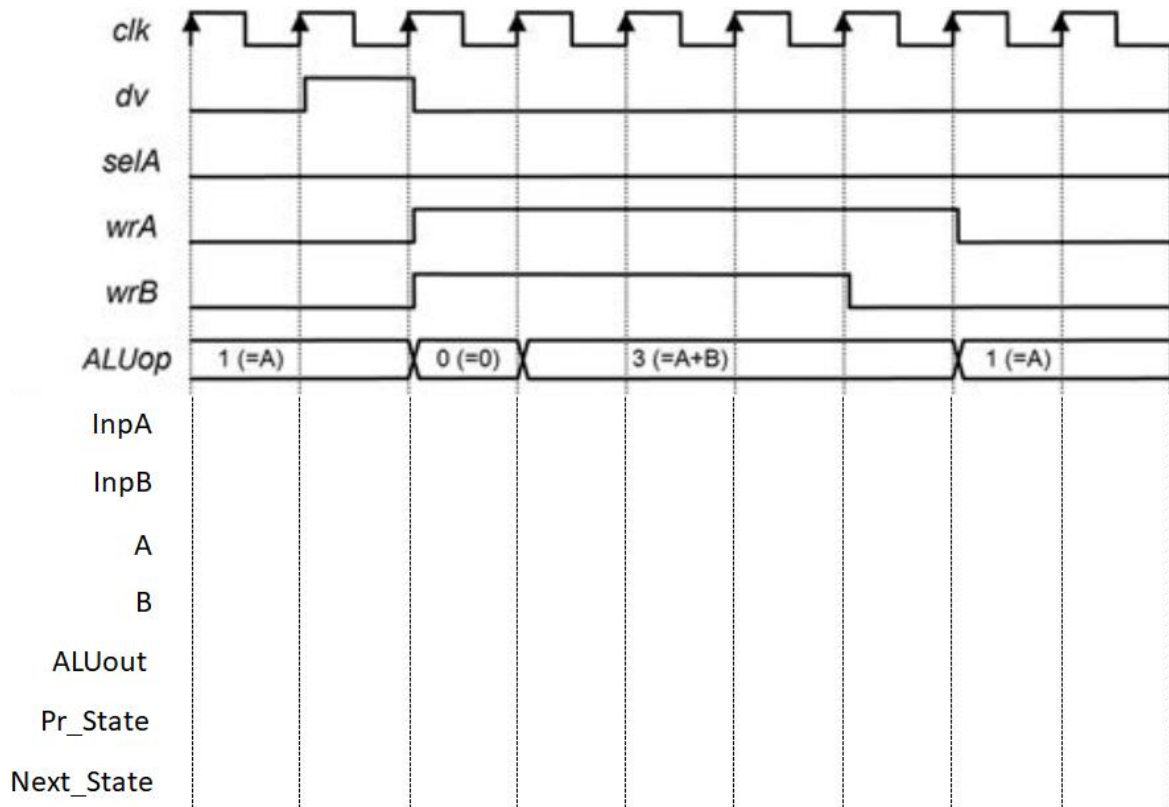
Remarque : ajoutez un reset *rst* synchrone à votre diagramme de transition, *rst* qui met à 0 les deux registres internes *A* et *B* et met 0 sur *ALUout*.

3) Complétez les deux lignes *Pr_State* et *Next_State* (respectivement les états présent et futur de votre machine) au chronogramme en annexe 1 [4]

4) Combien de bascules D seront nécessaires au minimum pour implémenter cette machine ? Justifiez [2]

5) Ecrire en VHDL votre machine à états [10]

Annexe 1



Examen du Cours CE419

Calculatrice interdite

Première session année 2023 – 2024

Filière EIS – Statut Apprenti

Durée : 90 mn

Un seul document autorisé :

« Syntaxe VHDL essentielle »

Les nombres entre crochets [X] après les questions indiquent approximativement le temps en minute à consacrer à chaque question ; ces nombres donnent également une indication du barème.

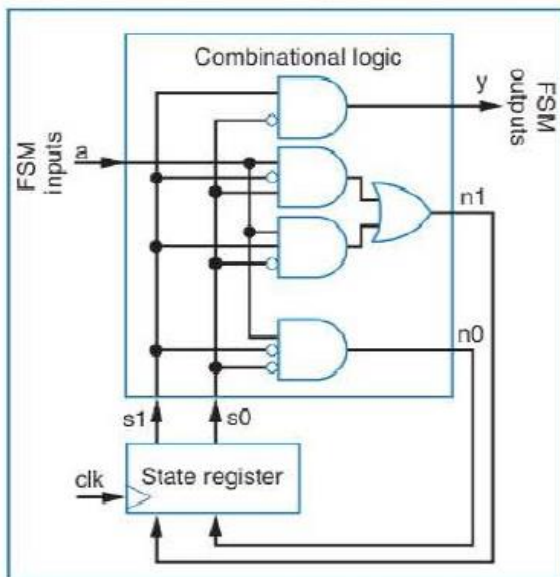
1 Reverse Engineering [42]

1.1 Le circuit ci-dessous est-elle une machine de Moore ou de Mealy ? Justifiez. [4]

1.2 Dessinez le schéma du diagramme de transitions de la machine ci-dessous [18]

1.3 Est-ce que tous les états de cette machine sont atteignables à partir d'une séquence et de n'importe quel autre état ? [4]

1.4 A partir de votre diagramme précédent, écrire le code VHDL de cette machine. Justifiez que la synthèse de votre code produira 2 DFF comme dans le circuit de départ [12+4].



2 ASMD comptage décomptage [50]

Nous allons concevoir un circuit permettant de compter le nombre de personnes dans une salle en utilisant la méthode à base d'ASMD vue en cours.

Le circuit a 2 entrées U et D chacune venant d'un bouton poussoir et un compteur 8 bits C. Pour chaque pression sur le bouton U le circuit incrémente (une seule fois !) C. Pour chaque pression sur le bouton D le circuit décrémente (une seule fois !) C. Si les 2 boutons sont pressés simultanément alors C ne change pas. Lorsque C a atteint sa valeur maximum ou 0 il ne reboucle pas à sa valeur 0 ou à sa valeur maximum. Ainsi il ne va pas plus haut que la valeur maximum sur 8 bits (non signé) et plus bas que 0. Une pression d'un bouton engendre le passage de 0 à 1 du signal associé ; la durée du 1 n'a pas d'importance.

2.1 Donnez l'ASM de ce circuit [18]

2.2 Donnez l'ASMD de ce circuit pour une machine de Moore [12]

2.3 Combien de bascules seront nécessaires pour réaliser son implémentation [4]

2.4 Donnez l'ASMD de ce circuit pour une machine de Mealy [12]

2.5 Combien de bascules seront nécessaires pour réaliser son implémentation [4]

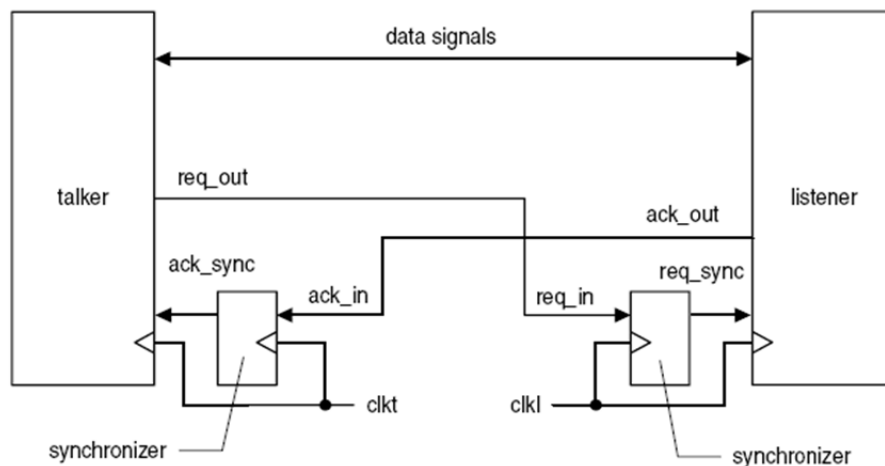
Examen du Cours CE410 Calculatrice interdite	Première session année 2017 – 2018 Filière EIS – Statut Etudiant Durée : 90 mn Un seul document autorisé : « Syntaxe VHDL essentielle »
--	---

Les nombres entre crochets [X] après les questions indiquent approximativement le temps en minute à consacrer à chaque question ; ces nombres donnent également une indication du barème.

1 Communication parallèle entre 2 FPGA [60]

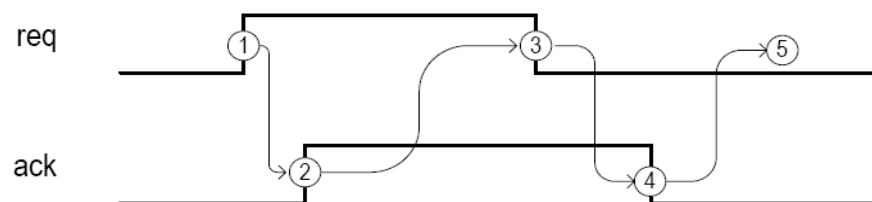
L'objectif est d'implémenter en VHDL une communication parallèle entre deux FPGA en utilisant la méthode de conception vue en cours à base d'ASMD.

La communication est basée sur l'architecture représentée sur la figure suivante. Le talker et le listener sont implémentés dans deux FPGA distincts.



Dans notre exemple *data* est un vecteur composé de 8 bits. Ce vecteur est envoyé du *talker* vers le *listener*.

Les signaux *req* et *ack* permettent de réaliser un échange de données en respectant le protocole poignée de main rappelé dans la figure ci-dessous.



1.1 Les synchroniseurs sont-ils utiles ici ? Expliquez [4].

Dans la suite pour simplifier on confondra *req_in* et *req_sync* avec *req*, ainsi que *ack_in* et *ack_sync* avec *ack* comme si ces deux synchroniseurs n'existaient pas.

1.2 Représentez les diagrammes de transitions des 2 machines à états de Moore permettant la gestion du protocole « poignée de main » dans le *talker* et le *listener* ci-dessus [2x6].

Vous considérerez :

Toute copie, modification, diffusion publique ou reproduction du contenu de ce document sans l'autorisation de l'auteur est interdite

- un signal reset *rst* permettant d'initialiser les deux machines dans un état de démarrage où les deux signaux *req* et *ack* sont à 0.
- un signal d'entrée *en* (enable) autorisant le *talker* et le *listener* à envoyer ou recevoir des données.

1.3 Représentez les chronogrammes (voir la liste des signaux ci-après) correspondant à l'envoi de 2 mots de 8 bits consécutifs ((1)₁₀ puis (2)₁₀) [22] (avec *en*=1 et *rst*=0).

Vous représenterez :

- *clkT* l'horloge du talker et *clkL* l'horloge du listener (2 horloges asynchrones, avec $T_{clkL}=2 \times T_{clkT}$).
- *req* et *ack*
- *data*
- les états présents des 2 machines à états du talker *Etp* et du listener *Elp*

Le talker doit envoyer le plus rapidement possible une série de 16 mots de 8 bits stockés dans un tableau *Data()*. Pour envoyer successivement ces 16 mots *Data(0)* *Data(1)*...puis *Data(15)*, vous utiliserez un compteur *count* sur 4 bits.

1.4 Représentez le diagramme de transitions de la machine à états de contrôle de l'ASMD incluant cette fois [16] :

- la commande de l'incrément du compteur *count*
- la commande pour l'envoi des données successives *Data(count)* vers *data*

Attention :

- utilisez pour les affectations non immédiates de simples flèches et pour les affectations immédiates des flèches doubles.

1.5 Combien de bascules au minimum seront générées lors de la synthèse de cette ASMD ? Détaillez votre calcul [6].

2 Filtrage numérique [32+4]

La description en annexe correspond à un filtre FIR.

2.1 Sur combien de bits sont codés les entrées et les coefficients [2] ? Combien de TAP contient ce filtre [2] ?

2.2 Faire un schéma de cette architecture en faisant apparaître le nom des signaux DIN, DOUT, MULT, COEFS, ADD [8].

2.3 S'agit-il d'une forme directe ou d'une forme transposée [2]? Quel est l'inconvénient principal de cette structure [2] ? Comment ici ce problème est en partie contourné [2]?

2.4 Sur combien de bits sont stockés les signaux dans la partie accumulation [2]? Expliquer pourquoi ce nombre de bits est suffisant [2]? Ce nombre de bits pourrait-il être réduit ? Expliquer [4] en donnant une équation permettant un nombre de bits optimums [2].

2.5 Combien de bascules seront générées par la synthèse de cette description [4] ?

2.6 Optionnel : En considérant les délais suivants calculez le chemin critique de cette structure [+4].

Tco	TAdd	Tsu
Temps de propagation des bascules	Temps de calcul d'un bit de l'additionneur	Set-up Time des bascules
500ps	100ps	1000ps

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

```
entity FIR is
```

```
Port (   CK      :      in      STD_LOGIC;
        DIN      :      in      STD_LOGIC_VECTOR (15 downto 0);
        DOUT     :      out     STD_LOGIC_VECTOR (37 downto 0));
end FIR;
```

```
architecture Behavioral of FIR is
```

```
signal DINR : std_logic_vector(DIN'range);
type COEFS_TYPE is array(59 downto 0) of std_logic_vector(15 downto 0);
constant COEFS : COEFS_TYPE := ( x"ffff",x"ffb3",x"00d6",x"0063",x"fdb1",x"01d9",x"025f",x"fab4",
x"0141",x"063b",x"f92d",x"fe5a",x"0825",x"fbe0",x"fccd",x"031e",
x"0011",x"0410",x"f766",x"fcc2",x"1730",x"eeb3",x"ea68",x"2dac",
x"f57a",x"cd9b",x"3731",x"0dd7",x"b74e",x"2a15",x"2a15",x"b74e",
x"0dd7",x"3731",x"cd9b",x"f57a",x"2dac",x"ea68",x"eeb3",x"1730",
x"fcc2",x"f766",x"0410",x"0011",x"031e",x"fccd",x"fbe0",x"0825",
x"fe5a",x"f92d",x"063b",x"0141",x"fab4",x"025f",x"01d9",x"fdb1", x"0063",x"00d6",x"ffb3",x"ffff");
type MULT_TYPE is array(59 downto 0) of std_logic_vector(31 downto 0);
signal MULT : MULT_TYPE;
type ADD_TYPE is array(59 downto 0) of std_logic_vector(37 downto 0);
signal ADD : ADD_TYPE;
constant ZERO : std_logic_vector(37 downto 0) := (others => '0');
begin
process(CK)
begin
if CK'event and CK = '1' then
    DINR <= DIN;
    for I in 59 downto 0 loop
        MULT(I) <= DINR * COEFS(59-I);
        if I = 0 then
            ADD(I) <= ZERO + MULT(0);
        else
            ADD(I) <= MULT(I) + ADD(I-1)
        end if;
    end loop;
    DOUT <= ADD(59);
end if;
end process;
end Behavioral;
```

Examen du Cours CE419 Calculatrice interdite Prénom : Nom :	Première session année 2021 – 2022 Filière EIS par apprentissage Durée : 90 mn Un seul document autorisé : « Syntaxe VHDL essentielle »
---	---

Les nombres entre crochets [X] après les questions indiquent approximativement le temps en minute à consacrer à chaque question ; ces nombres donnent également une indication du barème.

La lisibilité des schémas et des codes sera prise en compte dans la notation.

1 QCM partie cours HLS [12/90]

Une ou plusieurs bonnes réponses sont possibles par question.

2 points par questions

1.1 Les outils HLS permettent de générer rapidement plusieurs :

- A codes C
- B codes RTL
- C netlists
- D bitstreams

1.2 Les outils HDL permettent d'évaluer des architectures avec différents compromis _____

- A surface/performance
- B performance/rapidité
- C surface/coût
- D nombre de portes/fréquence maximale

1.3 Grâce aux outils HLS il est possible de :

- A générer plusieurs bitstreams rapidement
- B simuler les codes RTL en réutilisant les testbenchs écrits en C
- C simuler les codes C en réutilisant les testbenchs VHDL
- D mesurer la consommation de différents codes écrits en C

1.4 Le style du code C a

- A aucun impact sur le résultat
- B à un impact négligeable sur le résultat
- C à un impact majeur sur les résultats
- D un impact mais sur la forme uniquement du code RTL produit

1.5 Les outils HLS permettent de _____

- A obtenir des circuits de meilleures qualités que ceux générés manuellement
- B de réduire l'effort des concepteurs
- C d'explorer rapidement les caractéristiques de plusieurs architectures
- D de ne pas avoir à se soucier des problématiques architecturales

1.6 Les outils HLS permettent de _____

- A gérer automatiquement les problèmes de dépassement lors des calculs
- B d'utiliser des types C spécifiques pour définir des tailles de vecteurs précis
- C d'utiliser des nombres à virgules fixes
- D d'utiliser des nombres flottants

2 Accumulation dans des boucles imbriquées [79+10/90]

```

1 #include "accum.h"
2 #include <ac_int.h>
3 #define MAX 10
4 void accumulate(int din[2][4], int &dout){
5     int acc=0;
6     ROW:for(int i=0;i<2;i++){
7         if(acc>MAX)
8             acc = MAX;
9         COL:for(int j=0;j<4;j++){
10             acc += din[i][j];
11         }
12     }
13     dout = acc;
14 }

```

2.0 Pour une matrice *din* constituée d'éléments tous égaux à 1 combien vaudrait la sortie *dout* ? même question si tous les éléments valent 3 ? Justifiez vos réponses [5+5].

2.1 Donnez un ASM (organigramme) représentant graphiquement l'algorithme précédent [10]

2.2 Donnez un ASMD de l'ASM précédent, en utilisant une machine de Mealy pour la partie contrôle (ne pas chercher à l'optimiser pour le moment) [10]

Remarque :

- Ajoutez les signaux *start* et *ready*

2.3 Optimisez le temps de calcul de l'ASMD en modifiant votre ASM initial mais en conservant au moins 2 états distincts correspondant aux deux boucles *i* et *j* [10].

2.4 De combien d'additionneurs et de bascules votre implantation de l'ASMD précédent aura-t-elle besoin ? Justifiez [5+5]

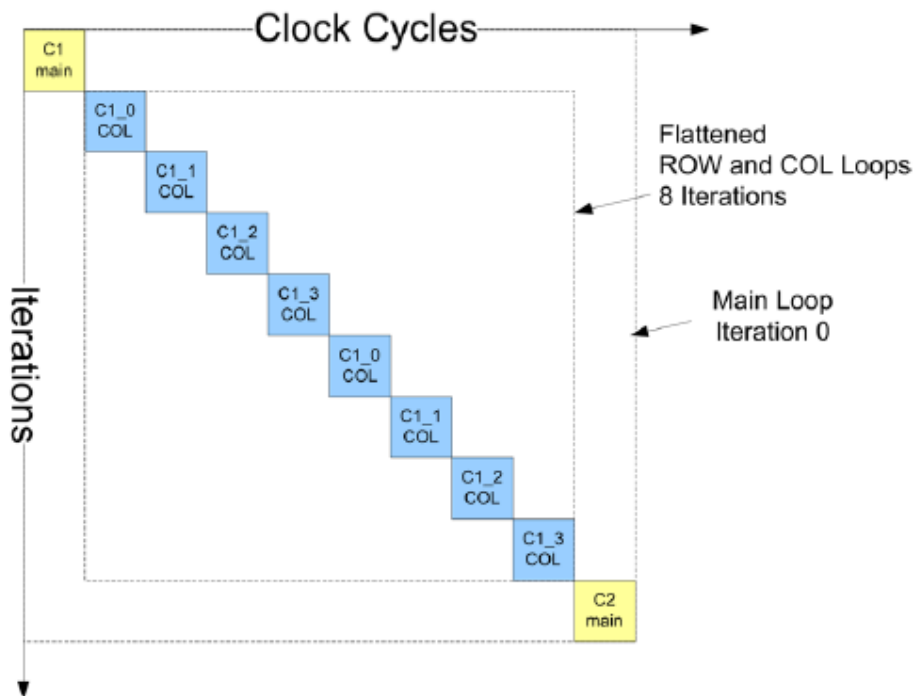
Remarque : on considèrera les valeurs stockées dans *din* sur *n* bits (on ne comptera pas les FF permettant de stocker les valeurs *din* en mémoire).

2.5 Faire un chronogramme du calcul représentant tous les signaux nécessaires à l'ASMD optimisé quand la matrice *din* est constituée uniquement d'éléments valant 3 [15].

2.6 Afin d'accélérer le calcul 2 solutions (offertes par les outils HLS) sont possibles ? Lesquelles ? [2+2].

2.7 Modifier l'algorithme initial pour appliquer une de ces 2 solutions [10]

2.8 Optionnel : Ci-dessous le résultat d'une de ces 2 solutions. Comparez avec le nombre de cycles que vous avez obtenus avec votre ASMD optimal. Donnez l'ASMD (avec une machine de Mealy) qui permettrait d'observer ce résultat [+10].



Conception de circuits numériques complexes sur FPGA (SN423)

Examen intermédiaire (30 minutes, sans supports)

Le sujet de cet examen intermédiaire porte sur la détection de l'appui sur un bouton poussoir (supposé sans rebond), à l'aide d'une machine à états. Le schéma électrique du câblage du bouton poussoir est représenté sur la figure ci-dessous. Plus particulièrement, le bouton est relié d'un côté à la masse et de l'autre à une résistance de tirage (*pull-up*), elle-même reliée à l'alimentation (Vcc).

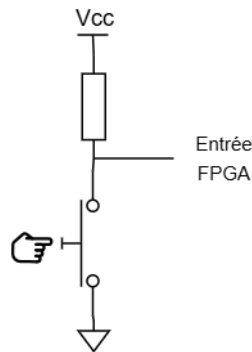


Schéma électrique du câblage du bouton poussoir

Pour détecter, que l'utilisateur a appuyé sur le bouton poussoir, il faudra mettre en place une chaîne de traitement, composé d'un synchroniseur et d'un détecteur de front descendant.

Partie 1 : Compréhension du schéma électrique (4 points)

Question 1 (2 points) : Représenter le chronogramme (d'une durée de 10 ms) du signal en entrée du FPGA, appelé *input*, lorsque qu'un utilisateur appuie sur le bouton au bout de 5 ms.

Question 2 (2 points) : Pour détecter l'appui sur le bouton, justifier de la nécessité d'utiliser un détecteur de front descendant (côté FPGA), et non d'un détecteur de front montant.

Partie 2 : Synchroniseur (4 points)

Question 3 (2 points) : Rappeler l'intérêt d'un synchroniseur dans ce cas d'utilisation.

Question 4 (2 points) : Donner le schéma électrique du synchroniseur (double).

Partie 3 : Machine de Moore (6 points)

Question 5 (3 points) : Donner le diagramme d'états du détecteur de front descendant, à l'aide d'une machine de Moore (sorties synchrones).

Question 6 (3 points) : Donner le diagramme ASM équivalent.

Partie 4 : Machine de Mealy (6 points)

Question 8 (3 points) : Donner le diagramme d'états du détecteur de front descendant, à l'aide d'une machine de Mealy (sorties asynchrones).

Question 9 (3 points) : Donner le diagramme ASM équivalent.

Conception de circuits numériques complexes sur FPGA (SN423)

Examen final (2 heures, sans calculatrice)

Exercice 1 (8 points) : Détection d'appui sur un bouton poussoir

Le but de cet exercice porte sur la détection de l'appui sur un bouton poussoir (supposé d'abord sans rebond, puis avec rebond), à l'aide d'une machine à états. Le schéma électrique du câblage du bouton poussoir est représenté sur la figure ci-dessous. Plus particulièrement, le bouton est relié d'un côté à la masse et de l'autre à une résistance de tirage (*pull-up*), elle-même reliée à l'alimentation (Vcc).

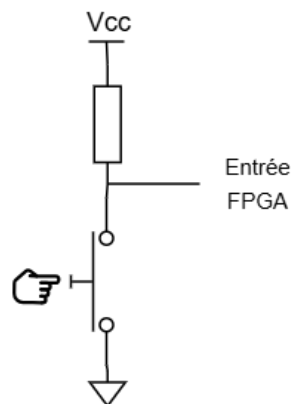


Schéma électrique du câblage du bouton poussoir

Pour détecter, que l'utilisateur a appuyé sur le bouton poussoir, il faudra mettre en place une chaîne de traitement, composé d'un synchroniseur et d'un détecteur de front descendant.

Question 1 (0.5 point) : Représenter le chronogramme (d'une durée de 10 ms) du signal en entrée du FPGA, appelé *input*, lorsque qu'un utilisateur appuie sur le bouton au bout de 5 ms.

Question 2 (0.5 point) : Pour détecter l'appui sur le bouton, justifier de la nécessité d'utiliser un détecteur de front descendant (côté FPGA), et non d'un détecteur de front montant.

Question 3 (0.5 point) : Rappeler l'intérêt d'un synchroniseur dans ce cas d'utilisation.

Question 4 (0.5 point) : Donner le schéma électrique du synchroniseur (double).

Question 5 (1.5 point) : Donner le diagramme d'états du détecteur de front descendant, à l'aide d'une machine de Mealy (sorties asynchrones).

Question 6 (1.5 point) : Donner le diagramme ASM équivalent.

Question 7 (1 point) : En déduire le circuit correspondant au détecteur de front descendant.

Question 8 (0.5 point) : En considérant que l'horloge du FPGA est à 100 MHz et que la durée des rebonds lors d'un appui est de 1 ms, déterminer le nombre de cycle que cela représente.

Question 9 (1.5 points) : Proposer un algorithme (avec interface et synchronisation) pour gérer la gestion des rebonds.

Bonus (1 point) : Donner l'ASMD à sorties asynchrones (Mealy) de votre algorithme.

Exercice 2 (7 points) : Exponentiation

Le but de cet exercice est de réaliser un circuit, basé sur une FSM obtenue à l'aide de la méthodologie ASMD, de la fonction réalisant l'exponentiation d'un nombre entier (naturel) **a**, par un autre nombre entier (naturel) **b** : **a^b**. L'algorithme récursif de l'exponentiation est le suivant :

$$f(a, b) = \begin{cases} a \times f(a, b - 1) & \text{si } a > 0 \\ 1 & \text{si } a = 0 \end{cases}$$

Question 1 (1 point) : Donner l'algorithme itératif correspondant à l'algorithme récursif ci-dessus.

Question 2 (1 point) : Donner l'algorithme (sans interface) correspondant à l'algorithme itératif.

Question 3 (1 point) : Donner l'algorithme (avec interface) correspondant à l'algorithme itératif. Vous considérerez que le circuit à une entrée **start** pour démarrer le calcul et une sortie **done** pour indiquer la fin du calcul.

Question 4 (2 points) : Donner l'ASMD à sorties synchrones (Moore) de l'algorithme.

Question 5 (2 points) : Donner l'ASMD à sorties asynchrones (Mealy) de l'algorithme.

Bonus (1 point) : Dans le cas du calcul de $f(3, 10)$, combien de cycles seront nécessaires pour obtenir le résultat de chaque FSM ? Justifier, à l'aide de ces résultats, quelle FSM est la plus rapide.

Exercice 3 (5 points) : Filtre FIR et HLS

Le but de cet exercice est de réaliser une implémentation itérative d'un filtre FIR. Pour cela, on considérera le cas d'un circuit fonctionnant à 100 MHz, avec une fréquence des échantillons à 2 MHz. Ainsi, il y a plusieurs cycles d'horloges entre chaque échantillon d'entrée, permettant de mettre en place une implémentation itérative du filtre FIR. Pour rappel, la formule d'un filtre FIR est la suivante :

$$y[n] = \sum_{k=0}^{K-1} w[k]x[n-k]$$

Avec :

- $y[n]$: Le n-ième échantillon de sortie du filtre FIR
- $x[n]$: Le n-ième échantillon d'entrée du filtre FIR
- $w[k]$: Le k-ième coefficient du filtre FIR
- K : Le nombre de coefficients du filtre FIR

Question 1 (1 point) : Combien il y a de cycles entre chaque échantillon d'entrée ?

Question 2 (1 point) : En considérant qu'une multiplication suivie d'une addition prend un cycle d'horloge, donner un majorant (réaliste) de la taille maximale du filtre implémentable.

Question (3 points) : Donner le code C permettant de générer l'implémentation itérative du filtre FIR. On considérera que les traitements sont effectués à l'aide de nombres à virgule flottante (float) et que le prototype de la fonction est le suivant (avec x_in : le nouvel échantillon d'entrée, x_reg : un tableau contenant les K derniers échantillons et w : un tableau contenant les K coefficients) :

```
float FIR_filter(float x_in, float *x_reg, const float *w, size_t K)
```

Conseil : Pensez à mettre à jour le contenu du tableau x_reg avec le nouvel échantillon x_in , en considérant que $x_reg[0]$ contient l'échantillon le plus récent et $x_reg[K-1]$ le plus ancien.

Bonus (1 point) : Quel est l'avantage de cette implémentation par rapport aux autres vues en cours ?